

Data Pipeline Sanity Checks

INDEX

1. Overview of Data Pipeline Sanity Checks

2. Core Pillars & Test Specifications

- 2.1 Volume & Completeness Validation
- 2.2 Schema Integrity & Data Type Safety
- 2.3 Uniqueness & Deduplication (Idempotency)
- 2.4 Business Logic & Data Reconciliation
- 2.5 Pipeline SLA & Data Freshness

3. Cloud Spanner Sanity Checks

- 3.1 Verify Table Schema & Record Existence
- 3.2 Data Integrity & Constraint Validation

4. BigQuery External Connection Sanity Checks

- 4.1 Test Spanner External Connection (EXTERNAL_QUERY)
- 4.2 Live Federated Fetch vs Direct Spanner Counts

5. BigQuery Gold Layer Aggregation Sanity Checks

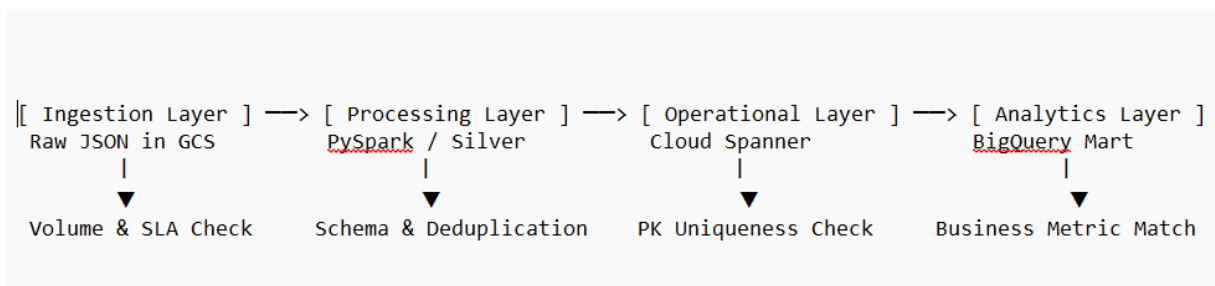
- 5.1 Inspect Materialized Daily Summary Table
- 5.2 Reconciliation Test — Gold Layer vs Live Federated Aggregation

1. Overview: Data Pipeline Sanity Checks:

Data pipeline sanity checks are quick, automatic health checks you run after deploying your project. They make sure your data is correct, complete, and moving safely through every step of your pipeline—from Google Pub/Sub and Cloud Storage to Cloud Spanner and Big Query.

Instead of testing every single line of code, sanity checks focus on the most important basics. They quickly catch big issues like missing data, duplicate records, broken formats, or stuck pipelines before bad data reaches end-users or dashboard reports.

2. Core Pillars of Pipeline Sanity Validation



Core Pillars & Test Specifications

2.1 Volume & Completeness Validation

- **Purpose:** Ensures raw records are landing continuously without silent drops, partial batch loads, or empty upstream extractions.
- **Specific Tests Executed:**
 - **Zero-Row Check:** Confirms upstream jobs processed records and target tables are populated ($\text{COUNT}(\ast) > 0$).
 - **Row Count Reconciliation:** Verifies record count parity across processing layers (e.g., PySpark cleaned records vs. Cloud Spanner inserts).
 - **Null Value Audit:** Validates that mandatory business keys (`order_id`, `product_id`, `order_date`) contain zero NULL entries.

2.2 Schema Integrity & Data Type Safety

- **Purpose:** Guarantees structural correctness, confirming numeric fields and dates parse accurately without unhandled exceptions.
- **Specific Tests Executed:**
 - **Type Casting Check:** Verifies numeric attributes like quantity cast correctly as integers rather than string primitives.
 - **Timestamp Format Check:** Confirms date fields parse to valid UTC ISO timestamps (YYYY-MM-DD HH:MM:SS) without falling back to epoch defaults (1970-01-01).
 - **Schema Evolution Check:** Ensures upstream field additions do not break downstream writes or federated reads.

2.3 Uniqueness & Deduplication (Idempotency)

- **Purpose:** Validates that streaming re-runs or duplicate message arrivals do not create duplicate line items in downstream layers.
- **Specific Tests Executed:**
 - **Primary Key Constraint Check:** Ensures composite key pairs (order_id + product_id) maintain 100% uniqueness with zero duplicates (HAVING COUNT(*) > 1 yields 0 rows).
 - **Idempotency Check:** Confirms re-running PySpark or BigQuery jobs updates existing records instead of creating duplicate entries.

2.4 Business Logic & Data Reconciliation

- **Purpose:** Confirms mathematical consistency and rule enforcement across distinct database engines (OLTP vs. OLAP).
- **Specific Tests Executed:**
 - **Range Checks:** Verifies metric values align with business logic (e.g., quantity > 0).
 - **Aggregation Accuracy:** Reconciles metrics between layers—ensuring SUM(quantity) in Cloud Spanner matches SUM(total_items_sold) in the BigQuery summary mart.

2.5 Pipeline SLA & Data Freshness

- **Purpose:** Detects frozen scheduled queries, stale caches, or upstream pipeline latency before business decision-makers view outdated figures.
- **Specific Tests Executed:**
 - **Data Freshness Check:** Evaluates MAX(aggregated_at) against CURRENT_TIMESTAMP() to detect delays exceeding the 24-hour SLA.

- **Connectivity & Zero-ETL Handshake:** Confirms BigQuery queries live Cloud Spanner records via federated EXTERNAL_QUERY without IAM or network timeout errors.

3.Cloud Spanner Sanity Checks

*Execute these inside **Spanner Studio** (ecom-spanner-instance →ecom-prod-db).*

3.1 Check 1: Verify Table Schema & Record Existence

-- 1. Check total rows written by PySpark

```
SELECT COUNT(*) AS total_line_items  
FROM OrdersLineItems;
```

-- 2. Inspect latest 5 records

```
SELECT  
    order_id,  
    user_id,  
    product_id,  
    quantity,  
    order_date  
FROM OrdersLineItems  
ORDER BY order_date DESC  
LIMIT 5;
```

- **Expected Result:** total_line_items > 0 and column values match parsed bronze JSON records.

3.2-Check 2: Data Integrity & Constraint Validation

-- Verify no NULL keys or invalid quantities exist

```
SELECT  
    COUNTIF(order_id IS NULL) AS null_order_ids,  
    COUNTIF(product_id IS NULL) AS null_product_ids,  
    COUNTIF(quantity <= 0) AS invalid_quantities  
FROM OrdersLineItems;
```

- **Expected Result:** All three counts return 0.

4. BigQuery External Connection Sanity Checks

Execute these inside BigQuery Studio (snappy-mapper-498509-e0).

4.1-Check 3: Test Spanner External Connection

```
(EXTERNAL_QUERY) SQL

-- Test connectivity and basic read permissions

SELECT *

FROM EXTERNAL_QUERY (

    "projects/snappy-mapper-498509-e0/locations/asia-south1/connections/spanner-ecom-conn-v2",

    "SELECT 1 AS connection_test;"

);
```

- Expected Result: Returns a single row with value 1 without permission or timeout errors.

4.2-Check 4: Live Federated Fetch vs Direct Spanner Counts

-- Verify BigQuery can read line items directly through the Spanner connection

```
SELECT

    COUNT(*) AS federated_row_count,

    COUNT(DISTINCT product_id) AS distinct_products

FROM EXTERNAL_QUERY (

    "projects/snappy-mapper-498509-e0/locations/asia-south1/connections/spanner-ecom-conn-v2",

    "SELECT product_id FROM OrdersLineItems;"

);
```

- Expected Result: federated_row_count matches the total count retrieved in Spanner Check 1

5. BigQuery Gold Layer Aggregation Sanity Checks

*Execute these inside **BigQuery Studio**.*

5.1-Check 5: Inspect Materialized Daily Summary Table

```
-- Inspect the latest metrics inserted by Airflow Task 2
SELECT
    product_id,
    total_items_sold,
    total_orders,
    aggregated_at
FROM `snappy-mapper-498509-e0.ecom_analytics.daily_product_summary`
ORDER BY aggregated_at DESC, total_items_sold DESC
LIMIT 10;
```

- **Expected Result:** Populated product rows with valid aggregation timestamps.

5.2-Check 6: Reconciliation Test (Gold Layer vs Live Federated Aggregation)

```
-- Compare BigQuery summary table totals against live
federated calculations
WITH live_federated_calc AS (
    SELECT
        product_id,
        SUM(quantity) AS expected_items_sold,
        COUNT(DISTINCT order_id) AS expected_orders
    FROM EXTERNAL_QUERY(
        "projects/snappy-mapper-498509-e0/locations/asia-south1/connections/spanner-ecom-conn-v2",
        "SELECT order_id, product_id, quantity
    FROM OrdersLineItems;"
    )
)
```

```

    )
    GROUP BY product_id
),
latest_materialized AS (
    SELECT
        product_id,
        total_items_sold,
        total_orders
    FROM `snappy-mapper-498509-
e0.ecom_analytics.daily_product_summary`
    QUALIFY ROW_NUMBER() OVER (PARTITION BY product_id ORDER
BY aggregated_at DESC) = 1
)
SELECT
    f.product_id,
    f.expected_items_sold,
    m.total_items_sold,
    (f.expected_items_sold - m.total_items_sold)
AS item_diff,
    (f.expected_orders - m.total_orders) AS order_diff
FROM live_federated_calc f
LEFT JOIN latest_materialized m
ON f.product_id = m.product_id;

```

- **Expected Result:** item_diff and order_diff are both 0 for all products.